

Taller Técnico: Creación de un Bloc de Noticias con React, Componentes y APIs

Consumo de REST API (JSONPlaceholder), Pruebas en Postman e Implementación de UI asistida por Inteligencia Artificial

OBJETIVO GENERAL

Capacitar al estudiante en el ciclo completo de integración de servicios en el Frontend, desde la validación e inspección de endpoints mediante herramientas de pruebas (Postman) hasta la construcción, modularización y consumo de datos dinámicos en un proyecto React. Se implementará un diseño estético avanzado haciendo uso de componentes reutilizables y prompts optimizados para IA.

1. ¿DE QUÉ SE TRATA ESTA API?

Para este proyecto utilizaremos un servicio REST simulado de alta confianza llamado JSONPlaceholder. El endpoint específico con el que trabajaremos es:

<https://jsonplaceholder.typicode.com/posts>

Este servicio expone una colección ficticia de 100 publicaciones de blogs o noticias en formato JSON. Cada publicación u objeto contiene de manera estricta la siguiente estructura de datos:

- id: Identificador único del post (numérico).
- userId: Identificador del autor asignado al post.
- title: El título de la noticia o artículo de prensa.
- body: El contenido textual completo o cuerpo de la noticia.

2. PASO 1: VERIFICACIÓN Y PRUEBAS EN POSTMAN

Antes de programar en nuestra app, debemos validar que la API responda correctamente. Siga estas instrucciones:

1. Abra Postman y cree un nuevo Request de tipo GET.
2. Pegue la URL del endpoint en la barra de direcciones.
3. Haga clic en 'Send' y analice la respuesta.
4. Verifique que el HTTP Status Code sea '200 OK' y el formato sea un arreglo [] de objetos JSON.

3. PASO 2: CREACIÓN DEL PROYECTO REACT Y COMPONENTES

Abra su terminal y ejecute los comandos estándar para inicializar su entorno de desarrollo moderno con Vite:

```
npm create vite@latest proyecto-bloc -- --template react
cd proyecto-bloc
npm install
npm run dev
```

Estructura de Componentes Reutilizables

Para cumplir con las buenas prácticas de arquitectura de software, modularizaremos la aplicación en el directorio `/src/components`:

- **Header.jsx**: Barra de navegación superior con el nombre del sistema (Bloc de Noticias), logo o caja de herramientas.
- **ListaNoticias.jsx**: Contenedor principal que se encargará de mapear y orquestar la renderización de la información.
- **TarjetaNoticia.jsx**: Componente atómico y reutilizable que recibe un objeto individual mediante 'props' y muestra el título, cuerpo de la noticia y un botón de interacción.

4. 🛠️ PASO 3: CONSUMO DE LA API (LLAMADO DINÁMICO)

En el componente principal o en `ListaNoticias.jsx`, utilizaremos los hooks nativos de React `useEffect` y `useState` para gestionar el estado de los datos asíncronos:

```
import { useState, useEffect } from 'react';

function ListaNoticias() {
  const [noticias, setNoticias] = useState([]);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(response => response.json())
      .then(data => setNoticias(data.slice(0, 12))) // Limitamos a 12
noticias por UX
      .catch(error => console.error('Error fetching:', error));
  }, []);

  return (
    <div className="grid-noticias">
      {noticias.map(noticia => (
        <TarjetaNoticia key={noticia.id} info={noticia} />
      ))}
    </div>
  );
}
```

5. 🎨 PASO 4: DISEÑO VISUAL ASISTIDO POR IA

Para lograr un diseño profesional e impactante sin perder tiempo en configuraciones complejas, los estudiantes usarán herramientas de Inteligencia Artificial (ej. ChatGPT,

Claude, Gemini) para generar estilos CSS modernos. Copie y adapte el siguiente prompt optimizado:

"Actúa como un diseñador UI/UX experto en CSS. Genera estilos modernos, limpios y atractivos para una aplicación web de 'Bloc de Noticias' construida en React. Necesito estilos para un Header corporativo, un grid responsivo que use CSS Grid o Flexbox, y tarjetas de noticias (.card) que tengan bordes redondeados, una sombra sutil (box-shadow), efectos hover suaves de elevación, fuentes tipográficas profesionales como Inter o Roboto, y una paleta de colores sofisticada (fondos claros, textos oscuros con buen contraste, y un color de acento azul o carmesí para los botones). Devuélveme solo el código CSS listo para pegar en index.css."

6. CRITERIOS DE EVALUACIÓN

- **Validación Inicial (20%):** Capturas de pantalla o demostración funcional de la petición GET exitosa en Postman.
- **Arquitectura React (30%):** Creación correcta del proyecto y separación clara de componentes reutilizables (Header, Lista, Tarjeta) mediante Props.
- **Consumo Dinámico (30%):** Uso adecuado de useState y useEffect para traer y renderizar los datos desde el endpoint real sin errores.
- **Estética y Acabado Profesional (20%):** Interfaz atractiva y responsiva lograda mediante la aplicación y personalización de las sugerencias de la IA.